Test Reporting, Quality Gates & Optimisation

Day 7 · Module 7 · 2 Hours

SDET / DevOps Engineer

Intermediate

40% Theory / 60% Hands-on

TOOLS

Allure Report 3

JaCoCo 0.8.14

LEARNING OUTCOMES

What You Will Achieve Today

Generate and interpret Allure test reports

Configure Allure 3 with JUnit 5 and TestNG, generate rich HTML reports with trend history, and publish them to a hosting target such as GitHub Pages via CI.

Enforce quality gates with JaCoCo coverage thresholds

Add JaCoCo 0.8.14 to your Maven build, configure minimum coverage rules (line + branch), and fail the build automatically when thresholds are not met.

Identify and remove CI pipeline bottlenecks

Measure stage durations, apply test sharding, dependency caching, and parallel job strategies to bring slow pipelines under a target threshold.

Allure Report 3 — Setup & Key Features

pom.xml — Allure JUnit 5 integration (adapter v2.x)

```
<!-- Allure Report 3 is the current platform;     -->
<!-- Java adapters still use the 2.x artifact line --
>
<dependency>
  <groupId>io.qameta.allure</groupId>
  <artifactId>allure-junit5</artifactId>
  <version>2.33.0</version>
  <scope>test</scope>
</dependency>

<!-- allure-maven plugin -->
<plugin>
  <groupId>io.qameta.allure</groupId>
  <artifactId>allure-maven</artifactId>
  <version>2.17.0</version>
</plugin>

<!-- Generate: mvn allure:report -->
<!-- Results:  target/allure-results/  -->
<!-- Report:   target/site/allure-maven-plugin/ -->
```

Allure 3 — Key Features

Rich HTML reports

Interactive dashboards: pass/fail/skipped breakdown, duration bars, step-level execution detail, screenshots, and attachments. Far richer than JUnit XML alone.

Trend history across runs

Allure tracks test stability over time when reports are published. Identify tests that consistently fail, recently became flaky, or regressed after a specific commit.

Annotations — @Epic, @Feature, @Story, @Step

Annotate tests with business context: @Feature("Login") groups tests by feature; @Step annotates method calls for drill-down in the report; @Severity labels criticality.

Quality Gate via allure-framework/allure-action

Allure 3 supports GitHub Action that posts PR summaries and — if quality gate settings are configured in allurerc.mjs — posts a pass/fail Check on the PR.

Allure 3 platform — 2.x Java adapters

Allure Report 3 is the current report platform. Java adapters (allure-junit5, allure-testng) still use the 2.x Maven artifact line. Both are actively maintained by Qameta.

Publishing Allure Reports in GitHub Actions

.github/workflows/ci.yml — Allure report with history

```yaml
  test-and-report:
    runs-on: ubuntu-latest
    permissions:
      pull-requests: write
      checks: write
      contents: write   # needed for gh-pages
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v5
        with:
          java-version: '21'
          distribution: temurin
      - uses: actions/cache@v4
        with:
          path: ~/.m2
          key: ${{ runner.os }}-m2-${{ hashFiles('**/pom.xml') }}

      - name: Run tests (generates allure-results/)
        run: mvn test -B
        if: always()        # collect results even on test failure

      # --- Allure 2 / gh-pages publish approach ---
      - name: Load Allure history from gh-pages
        uses: actions/checkout@v4
        if: always()
        continue-on-error: true
        with:
          ref: gh-pages
          path: gh-pages

      - name: Generate Allure Report with history
        uses: simple-elf/allure-report-action@v1.13
        if: always()
        with:
          gh_pages: gh-pages
          allure_history: allure-history
          allure_results: target/allure-results

      - name: Publish report to GitHub Pages
        uses: peaceiris/actions-gh-pages@v4
        if: always()
        with:
          github_token: ${{ secrets.GITHUB_TOKEN }}
          publish_branch: gh-pages
          publish_dir: allure-history
```

CI Reporting Notes

if: always() on every step

Ensures report generation and publish steps run even when tests fail — the report is most useful when there are failures to diagnose.

Trend history via gh-pages branch

Each run checks out the existing gh-pages branch, reuses prior report history from it, then publishes the updated report — enabling trend charts across runs.

Two CI patterns — choose one

Pattern A (left): simple-elf + peaceiris → publishes HTML to GitHub Pages. No PR annotation; only contents: write permission needed. Pattern B: allure-framework/allure-action → posts PR summaries + Checks (needs pull-requests: write + checks: write). Do not mix the two.

Allure report ≠ JUnit XML upload

Allure generates a full HTML report from allure-results/ JSON files. It is separate from upload-artifact (raw XML). Use both: JUnit XML for CI parsing, Allure for human-readable review.

JaCoCo 0.8.14 — Coverage Gates in Maven

Coverage thresholds are team heuristics, not universal standards. Risk-based guidance: 70–85% line coverage for typical code; 85–95% for critical business logic. 100% coverage is usually counterproductive — it leads to brittle tests that verify implementation details rather than behaviour.

pom.xml — JaCoCo 0.8.14 with LINE + BRANCH thresholds

```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.14</version>  <!-- Oct 2025, Java 25 support -->
  <executions>
    <!-- 1. Attach agent before tests run -->
    <execution>
      <goals><goal>prepare-agent</goal></goals>
    </execution>
    <!-- 2. Generate report after tests -->
    <execution>
      <id>report</id>
      <phase>verify</phase>
      <goals><goal>report</goal></goals>
    </execution>
    <!-- 3. Enforce coverage threshold -->
    <execution>
      <id>coverage-gate</id>
      <phase>verify</phase>
      <goals><goal>check</goal></goals>
      <configuration>
        <rules>
          <rule>
            <limits>
              <!-- Fail build if line coverage < 80% -->
              <limit>
                <counter>LINE</counter>
                <value>COVEREDRATIO</value>
                <minimum>0.80</minimum>
              </limit>
              <!-- Fail build if branch coverage < 70% -->
              <limit>
                <counter>BRANCH</counter>
                <value>COVEREDRATIO</value>
                <minimum>0.70</minimum>
              </limit>
            </limits>
          </rule>
        </rules>
      </configuration>
    </execution>
  </executions>
</plugin>

<!-- Run: mvn verify  -->
<!-- Report: target/site/jacoco/index.html -->
```

JaCoCo Key Concepts

prepare-agent goal

Attaches JaCoCo runtime agent via argLine before Surefire runs tests. Coverage data is written to target/jacoco.exec automatically.

LINE vs BRANCH coverage

LINE: % of source lines executed. BRANCH: % of decision branches taken (if/else, switch). Both matter — high line coverage can still miss untested branches.

check goal — fails the build

check runs in the verify phase after tests. If coverage is below minimum, Maven exits with BUILD FAILURE — same as a test failure. Blocks the merge gate.

Exclude generated / config code

Use <excludes> to skip generated classes, Spring config, DTOs. Coverage thresholds should measure business logic, not boilerplate that cannot be meaningfully tested.

v0.8.14 — Java 25 support

Released Oct 2025. Current stable release. Adds Java 25

CI Pipeline Optimisation Strategies

🔍 Diagnose before optimising. Check GitHub Actions job timings per step before applying fixes. The bottleneck is almost always in a specific step (slow test class, uncached dependencies, or sequential jobs that could run in parallel), not evenly spread across the pipeline.

Dependency Caching

Cache ~/.m2 (Maven) or ~/.gradle keyed on pom.xml hash. Eliminates 1–3 min of repeated network downloads on every run. Most impactful single optimisation for Java projects.

```
- uses:
actions/cache@v4
  with:
    path: ~/.m2
    key: ${{ runner.os
}}-m2- ${{
hashFiles('**/pom.xml')
}}
```

Fail-Fast Unit Tests First

Run unit tests (< 2 min) before integration tests. Add needs: unit-test to the integration-test job. Fast feedback: if unit tests fail, integration tests never start — saves compute.

```
jobs:
  unit-test:
    runs-on: ubuntu-
latest
  integration-test:
    needs: unit-test  #
gate
```

Test Sharding (Parallel Matrix)

Split integration tests across N runners using matrix: shard: [1,2,3,4]. Can substantially reduce wall-clock time when tests are independent and evenly distributed. Set fail-fast: false to see all shard results.

```
strategy:
  fail-fast: false
  matrix:
    shard: [1, 2, 3, 4]
# run: mvn verify -
Dshard.index=${{
#   matrix.shard }} -
Dshard.total=4
```

Parallel Jobs — Not Everything Sequential

Linting, SAST security scans, and API tests can often run in parallel with unit tests. Use needs: only where there is a real dependency. Unnecessary sequencing is a common source of slow pipelines.

```
jobs:
  lint:       # no needs
— runs in parallel
  unit-test: # no needs
— runs in parallel
  api-test:
    needs: unit-test  #
real dependency
```

Reading Allure Reports & Key CI Metrics

Allure Report Sections — What to Read

Overview dashboard

Total pass/fail/skip counts, pass rate %, and execution time. First stop after a CI run — tells you immediately whether the build is healthy or needs investigation.

Trend chart

Pass rate and failure count over the last N runs. A downward trend in pass rate signals deteriorating test stability. A spike in failures after a specific commit points to a regression.

Failures tab

Each failed test shows: full stack trace, step-by-step execution log, and any attached screenshots or logs. Read the step log top-down — find the first failing step, not just the final exception.

Categories — known flaky vs new failures

Allure categories let you classify failures: 'Product defects' (assertion failures), 'Test defects' (test code issues), 'Broken tests' (environment/config). Helps triage quickly.

Duration bars — find slow tests

Sort by duration to find the slowest tests. A single slow test class in a suite can dominate wall-clock time. Candidate for sharding or parallelisation.

Key CI Pipeline Metrics to Track

Test pass rate **Target > 95%**	Percentage of tests passing per run. Dips below 90% signal systemic stability issues.
Pipeline duration **Target < 10 min**	Total wall-clock time from push to green. Over 15 min discourages developers from pushing frequently.
Flake rate **Target < 2%**	Tests that fail without a code change. Track per-test and set SLAs for resolution.
Coverage % **Per team policy**	Line + branch coverage. Enforce via JaCoCo gate; report trend over time to avoid decay.
Mean time to green **Trend down**	Average time from test failure detection to green CI. Measures team responsiveness to test debt.

DEMO

Demo: Report → Gate → Optimise

01 Generating and Publishing an Allure Report

- Add allure-junit5 dependency and allure-maven plugin to pom.xml
- Annotate a test: @Feature("Login") @Story("valid credentials") @Severity(SeverityLevel.CRITICAL)
- Run mvn test — observe target/allure-results/ JSON files generated
- Run mvn allure:report — open target/site/allure-maven-plugin/index.html in browser

02 Configuring JaCoCo Coverage Thresholds

- Add jacoco-maven-plugin 0.8.14 to pom.xml with prepare-agent + report + check goals
- Set LINE minimum: 0.80, BRANCH minimum: 0.70 in the check execution
- Run mvn verify — observe coverage report at target/site/jacoco/index.html
- Drop coverage by commenting out a test — watch mvn verify fail with BUILD FAILURE

03 Identifying and Fixing a Pipeline Bottleneck

- Review Actions tab: click any completed job → expand step timings
- Identify the slowest step (likely: dependency download or slow integration test class)
- Add actions/cache@v4 for ~/.m2 — re-run and compare duration before vs after
- Split integration tests into 2 shards using matrix strategy — observe wall-clock improvement

HANDS-ON LAB

Lab: Report, Gate & Optimise

40% Theory | 60% Hands-on

Task 1

Allure report URL (GitHub Pages)

Generate Allure Report with Test Annotations

1. Add allure-junit5 v2.33.0 to pom.xml; configure allure-maven plugin
2. Annotate 3 existing tests: @Feature, @Story, and @Severity
3. Run mvn test && mvn allure:report — open index.html, verify annotations appear
4. Add the CI publish steps (simple-elf + peaceiris) to ci.yml; push and confirm report publishes

Task 2

CI build showing gate pass + failure evidence

Configure and Trigger a JaCoCo Coverage Gate

1. Add jacoco-maven-plugin 0.8.14 with prepare-agent + report + check goals
2. Set LINE minimum: 0.80 — run mvn verify and confirm gate passes
3. Comment out test method(s) to drop coverage — run mvn verify and confirm BUILD FAILURE
4. Restore tests — confirm gate passes again; add ci.yml step: mvn verify

Task 3

Before/after timing comparison note

Optimise Slow CI Pipeline

1. Record current pipeline duration from the Actions tab (baseline)
2. Add actions/cache@v4 for ~/.m2 — push and record new duration
3. Split integration-test job into 2 shards via matrix strategy
4. Record final duration; document before/after comparison in a short note

Day 7 Deliverables & Key Takeaways

Deliverables You Take Away

- Published Allure Test Report with trend history (GitHub Pages URL)
- CI Evidence — JaCoCo gate pass + deliberate failure screenshot

Key Takeaways

1. Allure reports give human-readable insight into failures, trends, and test health that raw JUnit XML cannot
2. JaCoCo check goal fails the Maven build automatically — coverage enforcement is a first-class quality gate
3. Use LINE + BRANCH coverage together; line coverage alone misses untested decision branches
4. Coverage thresholds are team heuristics: 70–85% for typical code; avoid targeting 100% (leads to brittle tests)
5. Diagnose before optimising — check step timings in Actions tab to find the actual bottleneck
6. Dependency caching and parallel jobs are the highest-return CI optimisations for most Java projects

Tomorrow — Day 8: End-to-End Continuous Testing Strategy & Capstone Preparation · Integrating all modules into a cohesive strategy

Hands-on: Capstone project brief · end-to-end pipeline design · strategy document · peer review

RECAP QUIZ

Module 7 – Quick Knowledge Check

Q1. Why do all Allure-related steps in the CI workflow use if: always()?

- Without if: always(), GitHub Actions skips subsequent steps when a previous step fails. If tests fail and the report step is skipped, there is no report to diagnose the failure — exactly when you need it most. always() ensures report generation and publishing run regardless of test outcome.

Q2. What is the difference between LINE coverage and BRANCH coverage in JaCoCo?

- LINE coverage measures what percentage of source code lines were executed during tests. BRANCH coverage measures what percentage of decision branches (if/else, switch cases) were taken. High line coverage can still miss untested branches — you need both metrics for meaningful quality gates.

Q3. A team sets a JaCoCo LINE coverage minimum of 0.90 (90%) and the build fails. Where do you look first?

- Open target/site/jacoco/index.html. Click through to the lowest-coverage packages and classes. Look for red/orange lines — these are uncovered. Check if any recently added classes lack tests, or if generated/config classes (which cannot be meaningfully tested) need to be excluded from the JaCoCo check rule.

Q4. You add Maven dependency caching to a CI job and the pipeline duration barely changes. What are the likely causes?

- The cache may not be hitting: check the cache key matches the actual path (~/.m2 for Maven). Alternatively, dependency download was not the bottleneck — use GitHub Actions step timings to find the actual slow step. Common real bottlenecks: a slow integration test class, a sequential job that could run in parallel, or a Docker image pull.